

Customize

For CSS-only changes, keep a built-in theme and append project CSS with styles in `calepin.toml`:

```
theme = "academic"
styles = [
  "styles/site.css",
  "styles/figures.css",
]
```

Configured CSS files load after the selected theme's CSS, in the order listed. Paths are resolved relative to `calepin.toml`, and each file must have a `.css` extension. The setting affects HTML output only.

In Calepin HTML output, `#title()` is rendered as the page `<h1>`, so the first Typst heading = becomes `<h2>`, == as `<h3>`, and so on. If you target heading selectors in CSS, use this mapping unless your theme remaps heading markup.

Style raw HTML with CSS

In some contexts, it can be useful to generate very simple HTML documents, and to define CSS to style raw classless HTML elements directly. One way to achieve this is to use the `typst` theme, which creates unstyled HTML that leaves styling fully in your project CSS. Set `theme = "typst"` when you do not want a bundled base theme. Configured styles still apply to HTML output:

```
theme = "typst"
styles = ["styles/raw.css"]
```

Override theme tokens

Built-in HTML themes expose stable CSS custom properties in the `--calepin-*` namespace. The recommended best practice is to override these tokens from project CSS rather than try to target the internals directly.

This override uses token variables for major surfaces and then applies your own typography and border styles, while leaving all Calepin component structure intact.

```
/* styles/custom.css */
:root {
  --calepin-color-background: #f6f7f9;
  --calepin-color-text: #111827;
  --calepin-color-accent: #2563eb;
  --calepin-color-border: #d1d5db;
}

body {
  font-family: Inter, system-ui, sans-serif;
  background: var(--calepin-color-background);
  color: var(--calepin-color-text);
}

pre {
  border: 1px solid var(--calepin-color-border);
}
```

Light and dark themes

To define distinct values for light and dark mode, use `html[data-theme="light"]` and `html[data-theme="dark"]` selectors in your project stylesheet:

```
html[data-theme="light"] {
  --calepin-color-background: #f6f7f9;
  --calepin-color-text: #111827;
  --calepin-color-accent: #2563eb;
}

html[data-theme="dark"] {
  --calepin-color-background: #101826;
  --calepin-color-text: #f8fafc;
  --calepin-color-accent: #60a5fa;
}
```

Calepin sets `html[data-theme="light"]` or `html[data-theme="dark"]` when the theme toggle is forced by user preference, explicit control, or URL/state storage. If no explicit mode is set, the browser preference is used for initial mode.

Case study: Tufte

In this case study, we build on top of the academic theme to replicate features of the popular Tufte CSS article style: serif typography, warm paper colors, restrained accents, sidenotes, margin figures, and code/output surfaces that match the page.

Reference files and rendered output can be viewed here:

- [calepin.toml](#)
- [tufte.css](#)
- [tufte.typ](#)
- [HTML](#)
- [PDF](#)

Start with a small `calepin.toml` next to the document:

```
theme = "academic"
styles = ["tufte.css"]
```

The first line keeps all of the built-in academic theme structure: the single-document HTML wrapper, the theme toggle, sidenotes, side figures, code styling, and dark-mode support. The second line appends a local stylesheet after the academic theme CSS, so the case study can customize tokens without copying or ejecting a full theme.

The local `tufte.css` file is intentionally small. It overrides the public `--calepin-*` tokens instead of targeting private theme internals:

```
:root, html[data-theme="light"] {
  --calepin-font-body: Palatino, "Palatino Linotype", "Book Antiqua", Georgia, serif;
  --calepin-font-heading: Palatino, "Palatino Linotype", "Book Antiqua", Georgia, serif;
  --calepin-surface-code: #ffffd0;
  --calepin-surface-output: #ffffd0;
  --calepin-surface: #ffffd0;
  --calepin-surface-muted: #fff8eb;
  --calepin-color-background: #fffff8;
  --calepin-color-text: #12110f;
  --calepin-color-muted: #5a5650;
  --calepin-color-accent: #7a2e2a;
  --calepin-color-accent-hover: #5f2520;
  --calepin-color-link: #7a2e2a;
  --calepin-color-link-hover: #5f2520;
}

html[data-theme="dark"] {
  --calepin-color-background: #18130d;
  --calepin-color-text: #f6efe8;
  --calepin-color-muted: #c5baa7;
  --calepin-color-accent: #d58a7f;
  --calepin-color-accent-hover: #f0b7ab;
  --calepin-color-link: #d58a7f;
  --calepin-color-link-hover: #f0b7ab;
  --calepin-surface-code: #1f1a14;
  --calepin-surface-output: #1f1a14;
  --calepin-surface: #1f1a14;
  --calepin-surface-muted: #2b241b;
}
```

The document source can then use the normal academic-theme elements:

`calepin.elements.sidenote` for margin notes, `calepin.elements.sidefigure` for margin figures, and regular executable code chunks for computed output. The stylesheet changes the feel of those elements, but the layout behavior still comes from the built-in theme.

From the case-study directory, render the HTML and PDF with:

```
cd docs/themes/examples/tufte
calepin compile tufte.typ --config calepin.toml --format html
calepin compile tufte.typ --config calepin.toml --format pdf
```

The config path matters because `styles = ["tufte.css"]` is resolved relative to `calepin.toml`. Keeping the config, stylesheet, and document together makes the example portable: copy the directory, run the same commands, and the academic theme plus Tufte overlay are applied in both rendered outputs.

List of tokens

Group	Token	Purpose
Colors	<code>--calepin-color-background</code>	Page background in normal UI surfaces.
	<code>--calepin-color-text</code>	Primary body text color.
	<code>--calepin-color-muted</code>	Muted/de-emphasized text.
	<code>--calepin-color-border</code>	UI border color.
	<code>--calepin-color-accent</code>	Primary accent color (links, buttons, highlights).
	<code>--calepin-color-accent-hover</code>	Accent color for hover states.
	<code>--calepin-color-accent-soft</code>	Soft tint derived from accent.
	<code>--calepin-color-accent-contrast</code>	Text/icon contrast color on accent backgrounds.
	<code>--calepin-color-link</code>	Default link color.
	<code>--calepin-color-link-hover</code>	Link hover color.
	<code>--calepin-color-focus</code>	Focus ring/text emphasis color.
	<code>--calepin-color-selection</code>	Text-selection background color.
	<code>--calepin-color-info</code>	Info state color.
	<code>--calepin-color-success</code>	Success state color.
	<code>--calepin-color-warning</code>	Warning state color.
	<code>--calepin-color-danger</code>	Danger/error state color.
	<code>--calepin-color-important</code>	Important state color.
	<code>--calepin-color-info-soft</code>	Translucent info variant.

Callouts	--calepin-color-success-soft	Translucent success variant.
	--calepin-color-warning-soft	Translucent warning variant.
	--calepin-color-danger-soft	Translucent danger variant.
	--calepin-color-important-soft	Translucent important variant.
	--calepin-callout-note-color	Callout tone for note blocks.
	--calepin-callout-tip-color	Callout tone for tip blocks.
	--calepin-callout-warning-color	Callout tone for warning blocks.
Surfaces	--calepin-callout-caution-color	Callout tone for caution blocks.
	--calepin-callout-important-color	Callout tone for important blocks.
	--calepin-surface	Primary surface color for cards/panels.
	--calepin-surface-muted	Muted surface tone.
	--calepin-surface-raised	Raised surface tone.
	--calepin-surface-inset	Inset surface tone.
	--calepin-surface-code	Code block background.
Typography	--calepin-surface-output	Code/output container background.
	--calepin-font-body	Primary text font stack.
	--calepin-font-heading	Heading font stack.
	--calepin-font-mono	Monospace font stack.
	--calepin-font-size	Base font size multiplier.
	--calepin-font-size-sm	Small text font size.

Spacing	--calepin-font-size-aside	Aside/sidenote font size.
	--calepin-line-height	Default line height for body text.
	--calepin-line-height-tight	Tighter line height for headings.
	--calepin-heading-weight	Default heading weight.
	--calepin-code-font-size	Code and preformatted font size.
	--calepin-space-xs	Extra-small spacing token.
	--calepin-space-sm	Small spacing token.
	--calepin-space-md	Medium spacing token.
	--calepin-space-lg	Large spacing token.
	--calepin-space-xl	Extra-large spacing token.
Layout	--calepin-space-2xl	2XL spacing token.
	--calepin-block-gap	Vertical block spacing.
	--calepin-inline-gap	Horizontal inline spacing.
	--calepin-section-gap	Vertical section spacing.
	--calepin-content-width	Max width for main content regions.
	--calepin-wide-width	Max width for wide content regions.
	--calepin-page-width	Computed page width for two-column layout.
	--calepin-margin-width	Sidenote/margin note width.
	--calepin-margin-gap	Gap between body and margin region.
	--calepin-padding-inline	Inline page padding.
	--calepin-topbar-height	Topbar height token.
	--calepin-sidebar-width	Desktop sidebar width.

	<code>--calepin-sidebar-mobile-width</code>	Mobile drawer width.
	<code>--calepin-shell-gap</code>	Gap between shell grid tracks.
	<code>--calepin-toc-indent</code>	Indent used by generated TOC trees.
Borders	<code>--calepin-border-width</code>	Border width used across theme.
	<code>--calepin-border</code>	Shorthand border style using <code>--calepin-color-border`</code> .
	<code>--calepin-radius-sm</code>	Small corner radius.
	<code>--calepin-radius-md</code>	Medium corner radius.
	<code>--calepin-radius-lg</code>	Large corner radius.
Shadows	<code>--calepin-shadow-card</code>	Default card shadow.
	<code>--calepin-focus-ring</code>	Focus ring style.
Layers	<code>--calepin-z-topbar</code>	Z-index for top bar.
	<code>--calepin-z-backdrop</code>	Backdrop and overlays.
	<code>--calepin-z-drawer</code>	Side drawer z-index.
	<code>--calepin-z-popover</code>	Popover/dialog z-index.
	<code>--calepin-z-modal</code>	Modal z-index.
Syntax	<code>--calepin-syntax-foreground</code>	Syntax text color for code blocks.
	<code>--calepin-syntax-background</code>	Syntax background color for code blocks.
	<code>--calepin-syntax-border</code>	Syntax border color derived from foreground/background.